Let's run through the sequence of events again, with the target network in play (we'll leave out the details of experience replay):

1. Initialize the Q-network with parameters (weights) $\theta_Q$ (read "theta Q").
2. Initialize the target network as a copy of the Q-network, but with separate parameters $\theta_T$ (read "theta T"), and set $\theta_T = \theta_Q$.
3. Use the epsilon-greedy strategy with the Q-network's Q values to select action $a$.
4. Observe the reward and new state $r_{t+1}$,$s_{t+1}$.
5. The target network's Q value will be set to $r_{t+1}$ if the episode has just been terminated (i.e., the game was won or lost) or to $r_{t+1} + \gamma \max Q_{\theta r}(S_{t+1})$ otherwise (notice the use of the target network here).
6. Backpropagate the target network's Q value through the Q-network (not the target network).
7. Every $C$ number of iterations, set $\theta_T = \theta_Q$ (i.e., set the target network's parameters equal to the Q-network's parameters).

Notice from figure 3.15 that the only time we use the target network, $\hat{Q}$, is to calculate the target Q value for backpropagation through the Q-network. The idea is that we update the main Q-network's parameters on each training iteration, but we decrease the effect that recent updates have on the action selection, hopefully improving stability.

**Figure 3.15. This is the general overview for Q-learning with a target network. It's a fairly straightforward extension of the normal Q-learning algorithm, except that you have a second Q-network called the target network whose predicted Q values are used to backpropagate through and train the main Q-network. The target network's parameters are not trained, but they are periodically synchronized with the Q-network's parameters. The idea is that using the target network's Q values to train the Q-network will improve the stability of the training.**